



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών (ΣΗΜΜΥ)

8ο εξάμηνο

Ψηφιακά Συστήματα VLSI

3η εργαστηριακή άσκηση

Δημήτρης Βατάλας

(Α.Μ.: 03121093)

Αριστοτέλης Γεώργιος Δρίτσας

(Α.Μ.: 03122127)

Αθήνα, Μάρτιος 2026

Contents

Εισαγωγή	2
MAC: Μονάδα Πολλαπλασιασμού με Συσσωρευτή (Multiplier Accumulator Unit)	2
ROM	3
RAM	5
Control Unit - Μονάδα Ελέγχου	6
Συνολικό Κύκλωμα - FIR φίλτρο	8

Εισαγωγή

Ένα FIR φίλτρο περιγράφεται γενικά από τη σχέση:

$$y[n] = \sum_{k=0}^M h[k]x[n-k] = h[0]x[n] + h[1]x[n-1] + \dots + h[M]x[n-M]$$

όπου:

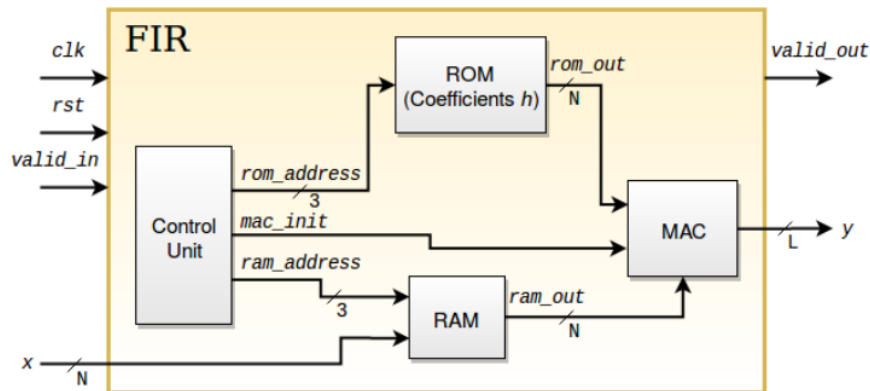
M: η τάξη του φίλτρου

y[n]: η διακριτή έξοδος τη χρονική στιγμή n

h[k]: ο k-οστός συντελεστής του φίλτρου

x[n]: η τιμή της εισόδου τη χρονική στιγμή n

Στην συγκεκριμένη εργαστηριακή άσκηση σύμφωνα με την εκφώνηση, θα υλοποιήσουμε ένα FIR φίλτρο με 8 taps. Αυτό αντιστοιχεί, σύμφωνα με την παραπάνω εξίσωση, σε φίλτρο τάξης $M = 7$ (δηλαδή με 8 συνολικούς όρους). Η τελική μορφή του κυκλώματος που θα προκύψει είναι η παρακάτω:



MAC: Μονάδα Πολλαπλασιασμού με Συσσωρευτή (Multiplier Accumulator Unit)

Το συγκεκριμένο δομικό στοιχείο του κυκλώματος αναλαμβάνει τον υπολογισμό των τιμών της εξόδου y, υλοποιώντας την πράξη:

$$a \leftarrow a + b \cdot x$$

Η μονάδα MAC (Multiply-Accumulate) δέχεται ως εισόδους τους σταθερούς συντελεστές του φίλτρου και τις διαδοχικές τιμές του σήματος εισόδου, ενώ στην έξοδό της παράγει κάθε φορά το αντίστοιχο δείγμα εξόδου y. Επιπλέον, περιλαμβάνει ένα σήμα ελέγχου, το οποίο χρησιμοποιείται για να σηματοδοτεί την έναρξη του υπολογισμού μιας νέας τιμής εξόδου και κατεπέκταση και της αρχικοποίησης της MAC.

Ιδιαίτερη σημασία έχει το απαιτούμενο εύρος της εξόδου y του φίλτρου. Πιο συγκεκριμένα, κάθε δείγμα εξόδου πρέπει να αναπαρίσταται με $2N + \log_2 N$ bits, αφού προκύπτει από το άθροισμα N γινομένων, όπου κάθε γινόμενο αντιστοιχεί σε πολλαπλασιασμό δύο N-bit αριθμών και έχει πλάτος έως 2N bits.

Στη συνέχεια παρουσιάζεται ο κώδικας VHDL, στον οποίο η συγκεκριμένη μονάδα υλοποιείται με behavioral αρχιτεκτονική.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity mac is
```

```

Port (
    a: in std_logic_vector (7 downto 0);
    b: in std_logic_vector (7 downto 0);
    clk: in std_logic;
    mac_init: in std_logic;

    y: out std_logic_vector (18 downto 0)
);

end mac;

architecture Behavioral of mac is

    signal accumulator: std_logic_vector(18 downto 0) := (others => '0');

begin

    process(clk)
    begin
        if rising_edge(clk) then
            if mac_init = '1' then
                accumulator <= "000" & (a * b);
            else
                accumulator <= accumulator + (a * b);
            end if;
        end if;

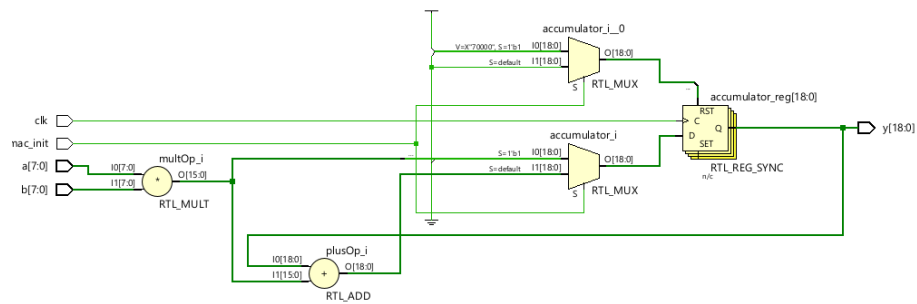
    end process;

    y <= accumulator;

end Behavioral;

```

Με χρήση του elaborated design έχουμε το παρακάτω αποτέλεσμα:



ROM

Η μνήμη ROM χρησιμοποιείται για την αποθήκευση των σταθερών συντελεστών του φίλτρου h . Ως είσοδο λαμβάνει μία διεύθυνση και επιστρέφει στην έξοδο την αντίστοιχη αποθηκευμένη τιμή, βέβαια αυτό συμβαίνει μόνο όταν το σήμα ενεργοποίησης (enable) βρίσκεται στη λογική τιμή 1. Οι συντελεστές του φίλτρου h που έχουν αποθηκευτεί, βάσει της εκφώνησης, είναι οι εξής:

$$h = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

Στη συνέχεια παρουσιάζεται ο κώδικας σε VHDL που περιγράφει τη λειτουργία της ROM:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity mlab_rom is
    generic (
        coeff_width : integer := 8 --- width of coefficients (bits)
    );
    Port (
        clk : in  STD_LOGIC;
        en : in  STD_LOGIC; --- operation enable
        addr : in  STD_LOGIC_VECTOR (2 downto 0); -- memory address
        rom_out : out STD_LOGIC_VECTOR (coeff_width-1 downto 0); -- output data
        rst : in  std_logic
    );
end mlab_rom;

architecture Behavioral of mlab_rom is

    type rom_type is array (7 downto 0) of std_logic_vector (coeff_width-1 downto 0);
    signal rom : rom_type:= ("00001000", "00000111", "00000110", "00000101", "00000100",
        "00000011", "00000010", "00000001"); -- initialization of rom with user data

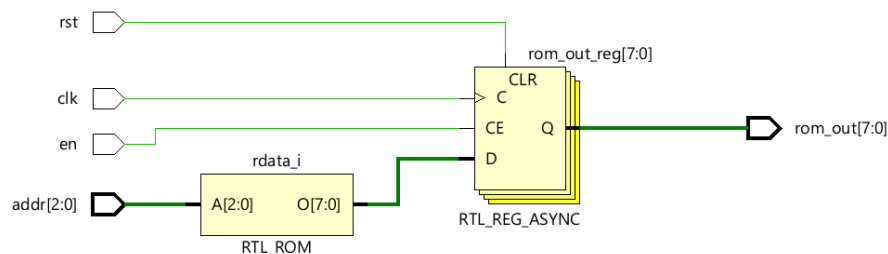
    signal rdata : std_logic_vector(coeff_width-1 downto 0) := (others => '0');
begin

    rdata <= rom(conv_integer(addr));

    process (clk, rst)
    begin
        if rst = '1' then
            rom_out <= (others => '0');
        elsif rising_edge(clk) then
            if en = '1' then
                rom_out <= rdata;
            end if;
        end if;
    end process;

end Behavioral;
```

Με χρήση του elaborated design έχουμε:



RAM

Η μνήμη RAM αξιοποιείται για την αποθήκευση της τρέχουσας τιμής του σήματος εισόδου, καθώς και ενός ιστορικού επτά προηγούμενων δειγμάτων, τα οποία είναι απαραίτητα για τον υπολογισμό της επόμενης τιμής εξόδου του φίλτρου. Κατά τη λειτουργία ανάγνωσης, λαμβάνει ως είσοδο μία διεύθυνση και επιστρέφει την αντίστοιχη αποθηκευμένη τιμή.

Κατά τη διαδικασία εγγραφής, η πιο πρόσφατη τιμή του σήματος τοποθετείται στην πρώτη θέση της μνήμης, ενώ οι ήδη αποθηκευμένες τιμές μετακινούνται κατά μία θέση, οδηγώντας στην απόρριψη της παλαιότερης τιμής.

Σημαντικό είναι ότι κατά την εγγραφή, έχει υλοποιηθεί κατάλληλα η RAM έτσι ώστε, η νέα τιμή που μόλις εγγράφηκε εμφανίζεται άμεσα στην έξοδο ταυτόχρονα με την αποθήκευσή της, αποφεύγοντας την εισαγωγή καθυστέρησης ενός κύκλου ρολογιού που θα προέκυπτε αν η ανάγνωση ακολουθούσε την εγγραφή.

Στη συνέχεια παρατίθεται ο κώδικας VHDL που περιγράφει τη λειτουργία της RAM:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity mlab_ram is
    generic (
        data_width : integer := 8 -- width of data (bits)
    );
    port (
        clk : in std_logic;
        we : in std_logic; -- memory write enable
        en : in std_logic; -- operation enable
        addr : in std_logic_vector(2 downto 0); -- memory address
        di : in std_logic_vector(data_width-1 downto 0); -- input data
        rst : in std_logic;
        do : out std_logic_vector(data_width-1 downto 0); -- output data
    end mlab_ram;

    architecture Behavioral of mlab_ram is

        type ram_type is array (7 downto 0) of std_logic_vector (data_width-1 downto 0);
        signal RAM : ram_type := (others => (others => '0'));
        signal tmp: std_logic;
    begin

        process (clk, rst)
        begin

            if rst = '1' then
                RAM <= (others => (others => '0'));
                do <= (others => '0');

            elsif clk'event and clk = '1' then
                if en = '1' then
                    if we = '1' then -- write operation
                        for i in 7 downto 1 loop
                            RAM(i) <= RAM(i-1);
                        end loop;
                        RAM(0) <= di;
                        do <= di;
                    else -- read operation
                        do <= RAM(conv_integer(addr));
                    end if;
                end if;
            end if;
        end process;
    end architecture Behavioral;
end mlab_ram;
```

```

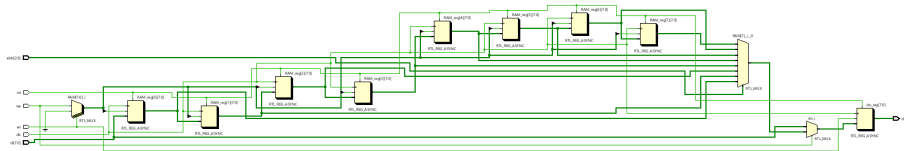
        end if;
    end if;

end process;

end Behavioral;

```

Με χρήση του elaborated design έχουμε:



Control Unit - Μονάδα Ελέγχου

Η Μονάδα Ελέγχου αναλαμβάνει τον συγχρονισμό του συστήματος, παρέχοντας σε κάθε κύκλο ρολογιού τις κατάλληλες διευθύνσεις προς τη RAM και τη ROM, καθώς και το σήμα που ενεργοποιεί την αρχικοποίηση της μονάδας MAC, οδηγώντας έτσι στην ομαλή συνεργασία όλων των επιμέρους συστημάτων. Στη συνέχεια παρουσιάζεται ο κώδικας σε VHDL που υλοποιεί τη μονάδα ελέγχου:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity control_unit is
    Port(
        valid_in: in std_logic;
        clk: in std_logic;
        rst: in std_logic;

        en: out std_logic;
        we: out std_logic;
        mac_init: out std_logic;
        rom_addr: out std_logic_vector(2 downto 0);
        ram_addr: out std_logic_vector(2 downto 0);
        valid_out: out std_logic
    );
end control_unit;

architecture Behavioral of control_unit is

    signal count: std_logic_vector(2 downto 0) := "000";
    signal enabled_flag: std_logic := '0';

begin

    process(clk, rst, valid_in)
    begin
        if rst = '1' then
            count <= "000";
            mac_init <= '1';

```

```

valid_out <= '0';
enabled_flag <= '0';
en <= '0';

elsif (enabled_flag = '0' and valid_in = '1') then
    enabled_flag <= '1';
    en <= '1';

elsif (rising_edge(clk) and enabled_flag = '1') then

    if valid_in = '1' and count = "000" then
        count <= count + 1;
        mac_init <= '1';
        valid_out <= '1';

    elsif valid_in = '1' and count > "000" then
        mac_init <= '0';
        valid_out <= '0';
        count <= "000";

    else
        mac_init <= '0';
        valid_out <= '0';
        count <= count + 1;

    end if;

end if;

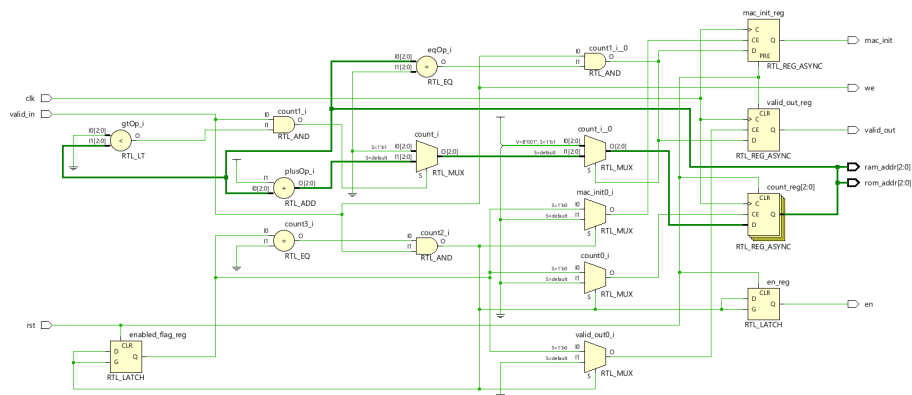
end if;
end process;

we <= valid_in;
rom_addr <= count;
ram_addr <= count;

end Behavioral;

```

Με χρήση του elaborated design έχουμε:



Συνολικό Κύκλωμα - FIR φίλτρο

Το συνολικό σύστημα προκύπτει από τη διασύνδεση των επιμέρους δομικών μονάδων που παρουσιάστηκαν προηγουμένως. Δεδομένου ότι ο συγχρονισμός και ο συντονισμός της λειτουργίας έχουν ήδη υλοποιηθεί μέσω της Μονάδας Ελέγχου, η τελική αρχιτεκτονική του συστήματος περιορίζεται ουσιαστικά στην κατάλληλη διασύνδεση των υπομονάδων μεταξύ τους.

Πιο συγκεκριμένα, οι μονάδες ROM, RAM και MAC συνδέονται μεταξύ τους σύμφωνα με το αρχικό σχηματικό που παραθέσαμε στην εισαγωγή, ενώ παράλληλα συνδέονται με τις εξωτερικές εισόδους και εξόδους του συνολικού κυκλώματος, το οποίο πλέον αντιμετωπίζεται ως ένα ενιαίο δομικό στοιχείο, δηλαδή το FIR φίλτρο. Η είσοδος του κυκλώματος τροφοδοτείται με νέα δεδομένα ανά 8 κύκλους ρολογιού, συνοδευόμενα από σήμα εγκυρότητας, το οποίο υποδηλώνει ότι η τιμή είναι έτοιμη προς επεξεργασία. Αντίστοιχα, το φίλτρο παράγει ένα νέο δείγμα εξόδου επίσης ανά 8 κύκλους ρολογιού, το οποίο συνοδεύεται από σήμα εγκυρότητας για τη διάκριση των έγκυρων αποτελεσμάτων από τις ενδιάμεσες τιμές.

Στη συνέχεια παρατίθεται ο κώδικας VHDL που περιγράφει τη λειτουργία του FIR φίλτρου σε επίπεδο structural αρχιτεκτονικής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity fir is
    Port (
        clk: in std_logic;
        rst: in std_logic;
        valid_in: in std_logic;
        x: in std_logic_vector (7 downto 0);

        valid_out: out std_logic;
        y: out std_logic_vector (18 downto 0)
    );
end fir;

architecture Behavioral of fir is

    component control_unit is
        Port (
            valid_in: in std_logic;
            clk: in std_logic;
            rst: in std_logic;

            en: out std_logic;
            we: out std_logic;
            mac_init: out std_logic;
            rom_addr: out std_logic_vector(2 downto 0);
            ram_addr: out std_logic_vector(2 downto 0);
            valid_out: out std_logic
        );
    end component;

    component mac is
        Port (
            a: in std_logic_vector (7 downto 0);
            b: in std_logic_vector (7 downto 0);
            clk: in std_logic;
            mac_init: in std_logic;
```

```

        y: out std_logic_vector (18 downto 0)
    );
end component;

component mlab_ram is
    generic (
        data_width : integer :=8 -- width of data (bits)
    );
    Port (
        clk : in std_logic;
        we : in std_logic; -- memory write enable
        en : in std_logic; -- operation enable
        addr : in std_logic_vector(2 downto 0); -- memory address
        di : in std_logic_vector(7 downto 0); -- input data
        rst : in std_logic;

        do : out std_logic_vector(7 downto 0) -- output data
    );
end component;

component mlab_rom is
    generic (
        coeff_width : integer :=8 --- width of coefficients (bits)
    );
    Port (
        clk : in STD_LOGIC;
        en : in STD_LOGIC; --- operation enable
        addr : in STD_LOGIC_VECTOR (2 downto 0); -- memory address

        rom_out : out STD_LOGIC_VECTOR (coeff_width-1 downto 0) ; -- output data
        rst : in std_logic
    );
end component;

signal rom_addr, ram_addr: std_logic_vector(2 downto 0):= (others => '0');
signal rom_data, ram_data: std_logic_vector(7 downto 0):= (others => '0');
signal rom_out, ram_out: std_logic_vector (7 downto 0) := (others => '0');
signal mac_init, en, we: std_logic;

begin
    rom_out <= rom_data when en = '1' else (others => '0');
    ram_out <= ram_data when en = '1' else (others => '0');

    cu: control_unit port map(
        clk => clk,
        rst => rst,
        en => en,
        we => we,
        valid_in => valid_in,
        rom_addr => rom_addr,
        ram_addr => ram_addr,
        mac_init => mac_init,
        valid_out => valid_out
    );

```

```

rom: mlab_rom port map(
    clk => clk,
    en => en,
    addr => rom_addr,
    rom_out => rom_data,
    rst=>rst
);

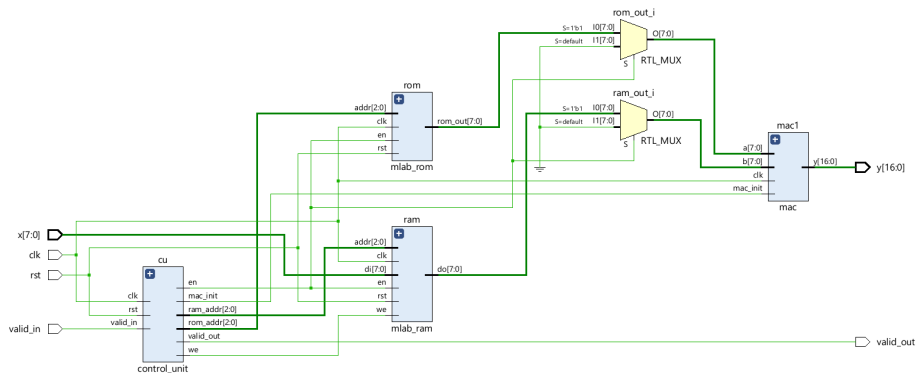
ram: mlab_ram port map(
    clk => clk,
    we => we,
    en => en,
    addr => ram_addr,
    di => x,
    rst => rst,
    do => ram_data
);

mac1: mac port map(
    clk => clk,
    a => rom_out,
    b => ram_out,
    mac_init => mac_init,
    y => y
);

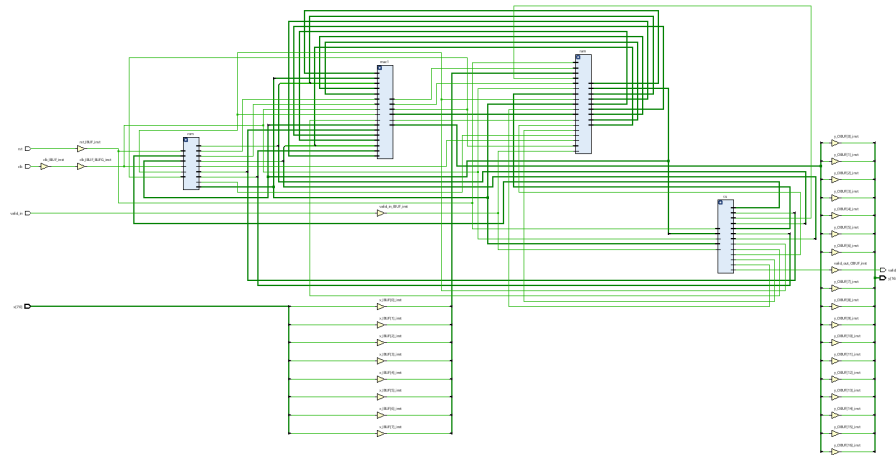
```

end Behavioral;

Με χρήση του elaborated design έχουμε:

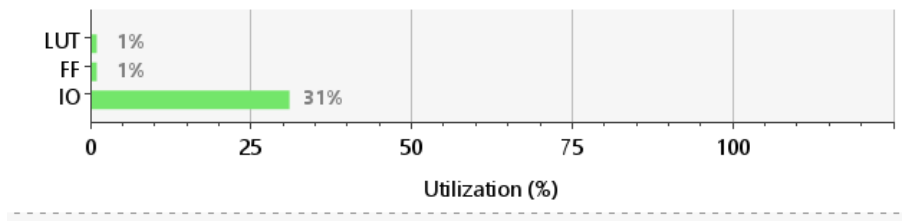


Με χρήση του post implementation έχουμε:

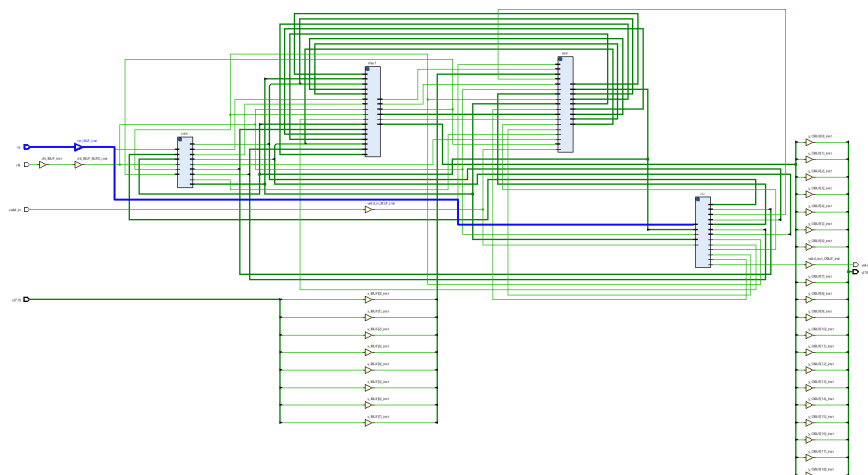


Στη συνέχεια παρουσιάζονται οι πόροι που χρησιμοποιήθηκαν για την υλοποίηση του κυκλώματος, όπως προκύπτουν μέσω της επιλογής *Utilization* στο synthesized design.

Resource	Utilization	Available	Utilization %
LUT	91	17600	0.52
FF	101	35200	0.29
IO	31	100	31.00



Εντοπίζουμε το παρακάτω κρίσιμο μονοπάτι με την βοήθεια του timing summary:



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clk
Path 21	∞	3	3	82	rst	cu/enabled_flag_reg/D	3.749	1.983	1.766	∞	input port clock	
Path 22	∞	1	1	82	rst	cu/enabled_flag_reg/CLR	2.169	0.961	1.209	∞	input port clock	

Παρατηρούμε ότι το κρίσιμο μονοπάτι παρουσιάζει καθυστέρηση ίση με 3.749 ns , που αντιστοιχεί σε μέγιστη συχνότητα 266.95MHz

Στη συνέχεια παρουσιάζεται το testbench που αναπτύχθηκε για την επαλήθευση της σωστής λειτουργίας του κυκλώματος. Έπειτα, παρατίθενται τα αποτελέσματα της προσομοίωσης, μαζί με σχολιασμό:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity fir_tb is
end fir_tb;

architecture behavior of fir_tb is

    component fir
        port (
            clk          : in  std_logic;
            rst          : in  std_logic;
            valid_in     : in  std_logic;
            x             : in  std_logic_vector(7 downto 0);
            valid_out     : out std_logic;
            y             : out std_logic_vector(16 downto 0)
        );
    end component;

    signal clk          : std_logic := '0';
    signal rst          : std_logic := '1';
    signal valid_in     : std_logic := '0';
    signal x            : std_logic_vector(7 downto 0) := (others => '0');
    signal valid_out     : std_logic;
    signal y            : std_logic_vector(16 downto 0);

    type input_type is array (0 to 31) of std_logic_vector(7 downto 0);

    signal input : input_type := (
        "11010000", "11100111", "01000000", "11101001",
        "10100001", "00011000", "01000111", "10001100",
        "11110101", "11110111", "00101000", "11111000",
        "11110101", "01111100", "11001100", "00100100",
        "01101011", "11101010", "11001010", "11110101",
        "00000000", "00000000", "00000000", "00000000",
        "00000000", "00000000", "00000000", "00000000",
        "00000000", "00000000", "00000000", "00000000"
    );

    constant CLK_PERIOD : time := 10 ns;

begin
```

```

    uut: fir
        port map (
            clk      => clk,
            rst      => rst,
            valid_in => valid_in,
            x        => x,
            valid_out => valid_out,
            y        => y
        );

    -- Clock generation
    clk_process : process
    begin
        while now < CLK_PERIOD * 1000 loop
            clk <= '0';
            wait for CLK_PERIOD / 2;
            clk <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process;

    -- Stimulus process
    stim_process : process
    begin
        -- Reset sequence
        rst <= '1';
        valid_in <= '0';
        wait for 10 * CLK_PERIOD ;

        rst <= '0';
        wait for 2 * CLK_PERIOD;

        -- Calculate again
        for i in 0 to 31 loop
            valid_in <= '1';
            x <= input(i);
            wait for CLK_PERIOD;

            valid_in <= '0';
            wait for 7 * CLK_PERIOD;
        end loop;

        wait;
    end process;

end behavior;

```

Αρχικά, εφαρμόζεται σήμα επαναφοράς (reset) στο κύκλωμα και στη συνέχεια εισάγονται οι παρακάτω τιμές στην είσοδο:

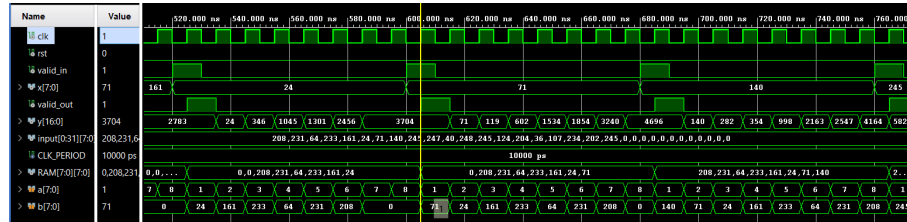
$$x = \{208, 231, 64, 233, 161, 24, 71, 140, 245, 247, 40, 248, 245, 124, 204, 36, 107, 234, 202, 245, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0\}$$

Υπολογίζουμε ότι η αναμενόμενη έξοδος είναι:

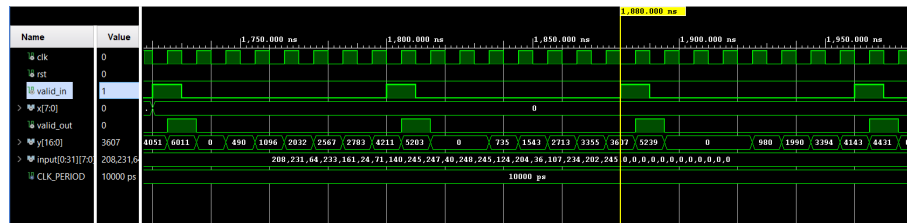
$$y = \{208, 647, 1150, 1886, 2783, 3704, 4696, 5828, 5333, 4670, 5319, 4631, 4603, 5771, 6696, 6965,$$

6256, 5518, 6598, 6011, 5203, 5239, 4431, 4931, 4756, 3331, 1960, 0, 0, 0, 0, 0, 0}

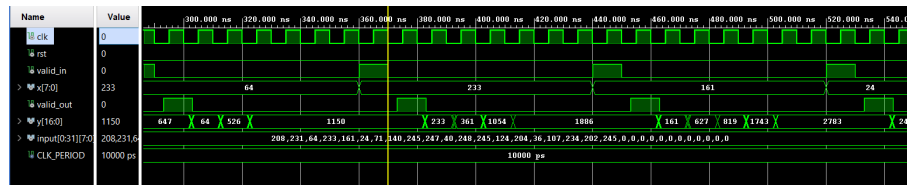
Τρέχουμε την προσομοίωση συμπεριφοράς (behavioural simulation) και παίρνουμε τα αποτελέσματα που φαίνονται στην παρακάτω εικόνα. Διαπιστώνουμε ότι το φίλτρο παράγει τις αναμενόμενες τιμές στην έξοδο. Το σήμα `valid_out`, το οποίο υποδεικνύει την εγκυρότητα της εξόδου, εμφανίζεται σωστά συγχρονισμένο με τις αντίστοιχες έγκυρες τιμές:



Στη συνέχεια τρέχουμε μια **functional** posti-implementation προσομοίωση και παρατηρούμε ότι όλα λειτουργούν κανονικά και το φίλτρο έχει την ζητούμενη συμπεριφορά.



Τέλος, τρέχουμε μια **timing** posti-implementation προσομοίωση, λαμβάνοντας υπόψη τις καθυστερήσεις των πυλών του FPGA και παρατηρούμε ότι και πάλι έχουμε τη συμπεριφορά που περιμέναμε.



Για να επιβεβαιώσουμε τα αποτελέσματα της προσομοίωσης φτιάχτηκε το παρακάτω script στην python:

```
def bin_to_int(b):  
    """Unsigned conversion"""  
    return int(b, 2)  
  
def fir_with_ram_print(inputs, coeffs):  
    n = len(coeffs)  
    ram = [0.0] * n # shift register  
  
    print("--- Demonstrating FIR Filter Usage ---\n")  
    print(f"Initial filter coefficients (h): {coeffs}")  
    print(f"Initial input array (X): {ram}\n")  
  
    for i, x in enumerate(inputs):
```

```

ram = [float(x)] + ram[:-1]

y = sum(ram[j] * coeffs[j] for j in range(n))

ram_str = " ".join(f"{v:.0f}." for v in ram)
print(f"Iteration {i} --- | [{ram_str}] | {y:.4f}")

# -----
# VHDL INPUT (UNSIGNED)
# -----

vhdl_inputs = [
    "11010000", "11100111", "01000000", "11101001",
    "10100001", "00011000", "01000111", "10001100",
    "11110101", "11110111", "00101000", "11111000",
    "11110101", "01111100", "11001100", "00100100",
    "01101011", "11101010", "11001010", "11110101",
    "00000000", "00000000", "00000000", "00000000",
    "00000000", "00000000", "00000000", "00000000",
    "00000000", "00000000", "00000000", "00000000"
]

inputs = [bin_to_int(b) for b in vhdl_inputs]

coeffs = [1, 2, 3, 4, 5, 6, 7, 8]

fir_with_ram_print(inputs, coeffs)

```

Και παίρνουμε τα παρακάτω αποτελέσματα:


```

Iteration 0 --- | [208. 0. 0. 0. 0. 0. 0. 0.] | 208.0000
Iteration 1 --- | [231. 208. 0. 0. 0. 0. 0. 0.] | 647.0000
Iteration 2 --- | [64. 231. 208. 0. 0. 0. 0. 0.] | 1150.0000
Iteration 3 --- | [233. 64. 231. 208. 0. 0. 0. 0.] | 1886.0000
Iteration 4 --- | [161. 233. 64. 231. 208. 0. 0. 0.] | 2783.0000
Iteration 5 --- | [24. 161. 233. 64. 231. 208. 0. 0.] | 3704.0000
Iteration 6 --- | [71. 24. 161. 233. 64. 231. 208. 0.] | 4696.0000
Iteration 7 --- | [140. 71. 24. 161. 233. 64. 231. 208.] | 5828.0000
Iteration 8 --- | [245. 140. 71. 24. 161. 233. 64. 231.] | 5333.0000
Iteration 9 --- | [247. 245. 140. 71. 24. 161. 233. 64.] | 4670.0000
Iteration 10 --- | [40. 247. 245. 140. 71. 24. 161. 233.] | 5319.0000
Iteration 11 --- | [248. 40. 247. 245. 140. 71. 24. 161.] | 4631.0000
Iteration 12 --- | [245. 248. 40. 247. 245. 140. 71. 24.] | 4603.0000
Iteration 13 --- | [124. 245. 248. 40. 247. 245. 140. 71.] | 5771.0000
Iteration 14 --- | [204. 124. 245. 248. 40. 247. 245. 140.] | 6696.0000
Iteration 15 --- | [36. 204. 124. 245. 248. 40. 247. 245.] | 6965.0000
Iteration 16 --- | [107. 36. 204. 124. 245. 248. 40. 247.] | 6256.0000
Iteration 17 --- | [234. 107. 36. 204. 124. 245. 248. 40.] | 5518.0000
Iteration 18 --- | [202. 234. 107. 36. 204. 124. 245. 248.] | 6598.0000
Iteration 19 --- | [245. 202. 234. 107. 36. 204. 124. 245.] | 6011.0000
Iteration 20 --- | [0. 245. 202. 234. 107. 36. 204. 124.] | 5203.0000
Iteration 21 --- | [0. 0. 245. 202. 234. 107. 36. 204.] | 5239.0000
Iteration 22 --- | [0. 0. 0. 245. 202. 234. 107. 36.] | 4431.0000
Iteration 23 --- | [0. 0. 0. 0. 245. 202. 234. 107.] | 4931.0000
Iteration 24 --- | [0. 0. 0. 0. 0. 245. 202. 234.] | 4756.0000
Iteration 25 --- | [0. 0. 0. 0. 0. 0. 245. 202.] | 3331.0000
Iteration 26 --- | [0. 0. 0. 0. 0. 0. 0. 245.] | 1960.0000
Iteration 27 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000
Iteration 28 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000
Iteration 29 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000
Iteration 30 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000
Iteration 31 --- | [0. 0. 0. 0. 0. 0. 0. 0.] | 0.0000

```